

Background: Shor's algorithm for the elliptic-curve discrete logarithm

Given a generator P of prime order r and $Q = [k]P$, recover the private scalar k

1. Encode k as a hidden period

Define a two-dimensional probe function over the elliptic-curve group:

$$f(u, v) = [u]P + [v]Q = [u + kv]P.$$

Because one additional copy of Q equals k copies of P ,

$$f(u - k, v + 1) = f(u, v).$$

Thus $(-k, 1)$ generates the hidden period subgroup. The double-scalar multiplication $[u]P + [v]Q$ is therefore the oracle whose periodicity encodes k ; Fourier sampling reveals a vector orthogonal to that period.

2. Coherent evaluation and Fourier sampling

Prepare two t -qubit scalar registers, with $M = 2^t \gtrsim r$, and a point accumulator initialized to $\mathcal{O}$:

$$\frac{1}{M} \sum_{u,v=0}^{M-1} |u, v\rangle |\mathcal{O}\rangle \xrightarrow{U_f} \frac{1}{M} \sum_{u,v=0}^{M-1} |u, v\rangle |[u]P + [v]Q\rangle.$$

Apply the two-register Fourier transform (or its semiclassical implementation), measure (s, t) , and classically recover and verify

$$t \equiv ks \pmod{r}, \quad k \equiv ts^{-1} \pmod{r}, \quad [k]P \stackrel{?}{=} Q.$$

The Fourier convention may reverse a sign; the recoverable modular relation is unchanged in substance.

3. Evaluate U_f by coherent windowed point additions

$$u = \sum_{j=0}^{\ell-1} u_j 2^{jw}, \quad v = \sum_{j=0}^{\ell-1} v_j 2^{jw}, \quad \ell = \left\lceil \frac{n}{w} \right\rceil.$$

$$[u]P + [v]Q = \sum_{j=0}^{\ell-1} [u_j]P_j + \sum_{j=0}^{\ell-1} [v_j]Q_j, \quad P_j = [2^{jw}]P, \quad Q_j = [2^{jw}]Q.$$

For each coherent w -bit address $z \in \{u_j, v_j\}$, a QROM table T_j supplies the selected precomputed point, the point is added to the accumulator, and the lookup workspace is uncomputed:

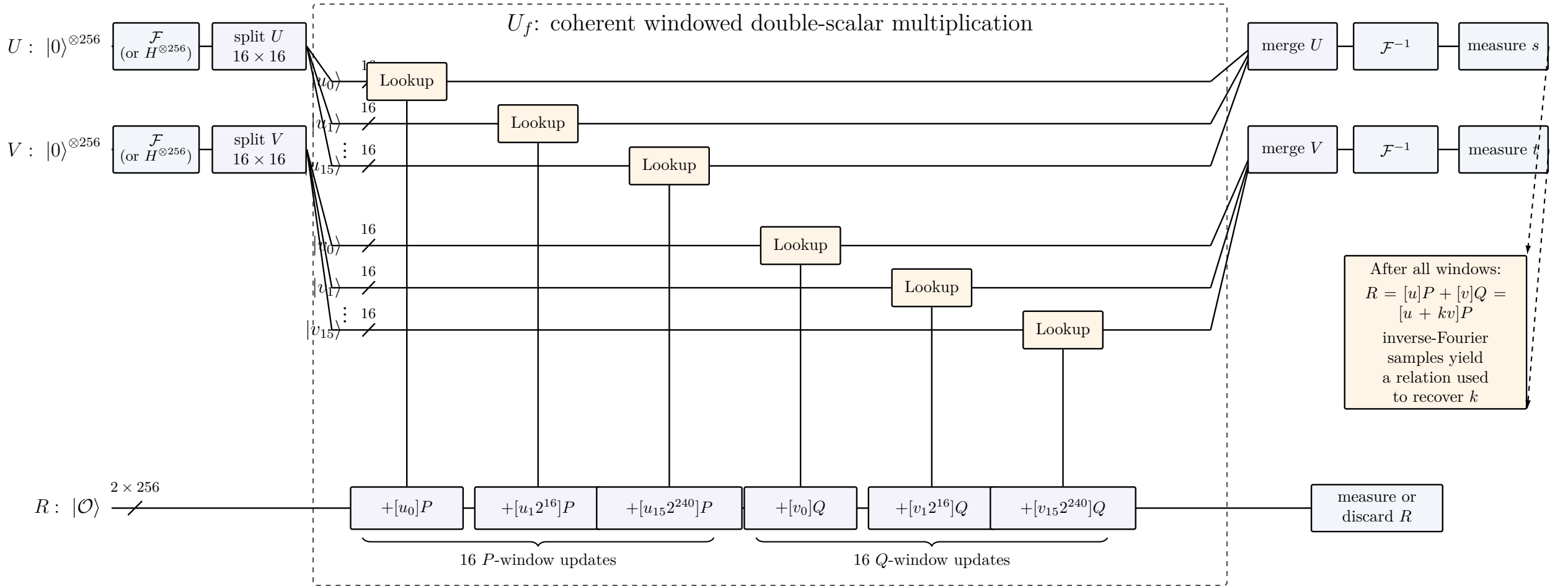
$$|z\rangle |0\rangle_A |R\rangle \xrightarrow{\text{lookup}} |z\rangle |T_j[z]\rangle_A |R\rangle \xrightarrow{\text{add}} |z\rangle |T_j[z]\rangle_A |R + T_j[z]\rangle \xrightarrow{\text{unlookup}} |z\rangle |0\rangle_A |R + T_j[z]\rangle.$$

The window register is preserved as a coherent address; it is not merely a single CCX-style control bit. For $n = 256$ and $w = 16$, there are $\ell = 16$ windows per scalar and 32 conceptual point-addition updates before boundary-level reductions.

Scope of this deck and submission 8e9c9a2. The analyzed source implements the in-place mixed-addition body $R \mapsto R + A$ for one classically known, temporarily materialized addend $A = (x_A, y_A)$. Under the challenge evaluator, it reaches $Q = 1,151$ peak logical qubits and $T = 1,299,453$ average executed Toffoli gates, giving $S = Q \times T = 1,495,670,403$. It does not implement the scalar registers, Fourier transforms, or QROM lookup/unlookup shown above. Embedding it into a fully coherent windowed-Shor circuit therefore requires a separately specified and validated lookup-use-unlookup interface for the selected point.

Shor's ECDLP circuit with the window controls expanded

Given P and $Q = [k]P$, evaluate $f(u, v) = [u]P + [v]Q = [u + kv]P$



The window registers pass through U_f unchanged: they are coherent QROM addresses controlling additions on R , not individual CCX-style controls.

For $n = 256$ and $w = 16$ there are 32 conceptual window updates; boundary optimizations reduce this to 28 point-addition calls in the paper's `secp256k1` estimate.

Conceptual fully coherent view; the semiclassical implementation interleaves Fourier operations with the windows and recycles the control qubits.

Generic in-place mixed addition: $R' = R + A$

Seven reversible register updates implement the affine group law while recycling X and Y .

In-place addition of the selected addend A

Input: accumulator $R = (x_R, y_R)$ and selected addend $A = (x_A, y_A)$ in $E(\mathbb{F}_p)$.

Generic-case assumptions: $x_R \neq x_A$ and $x_A \neq x_{R'}$.

Output: $R' = (x_{R'}, y_{R'}) = R + A$ is returned in the same X, Y registers.

1. $X \leftarrow x_R - x_A; \quad Y \leftarrow y_R - y_A$ $\triangleright X = d_x, Y = d_y$
2. $Y \leftarrow YX^{-1}$ $\triangleright Y = \lambda$
3. $X \leftarrow X + 3x_A$ $\triangleright X = x_R + 2x_A$
4. $X \leftarrow X - Y^2$ $\triangleright X = x_A - x_{R'}$
5. $Y \leftarrow YX$ $\triangleright Y = \lambda(x_A - x_{R'})$
6. $Y \leftarrow Y - y_A$ $\triangleright Y = y_{R'}$
7. $X \leftarrow x_A - X$ $\triangleright X = x_{R'}$

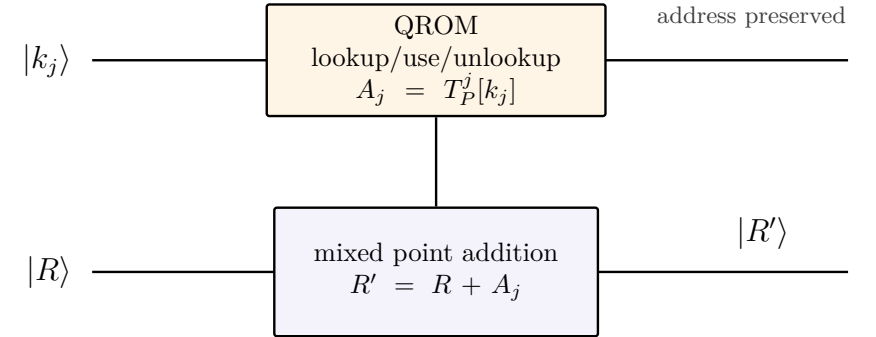
Algebraic check

$$\lambda = \frac{y_R - y_A}{x_R - x_A}, \quad x_{R'} = \lambda^2 - x_R - x_A, \quad y_{R'} = \lambda(x_A - x_{R'}) - y_A.$$

After Steps 3–4, $X = (x_R - x_A) + 3x_A - \lambda^2 = x_A - x_{R'}$; Steps 5–7 then recover $y_{R'}$ and $x_{R'}$ without allocating separate output-coordinate registers.

Use inside windowed Shor

The window word selects the addend A_j ; the lookup is used directly by the mixed-addition kernel and then uncomputed.



$A_j = (x_{A_j}, y_{A_j})$ is the elliptic-curve point selected for this call. For a w -bit window,

$$A_j = T_P^j[k_j], \quad T_P^j[i] = [i 2^{jw}]P$$

for a P -scalar window, with an analogous table T_Q^j for the public-point scalar.

The complete coherent map is

$$|k_j\rangle |R\rangle |0\rangle_{\text{work}} \mapsto |k_j\rangle |R'\rangle |0\rangle_{\text{work}}, \quad R' = R + T_P^j[k_j].$$

For $n = 256$ and $w = 16$, each scalar contains 16 such windows.

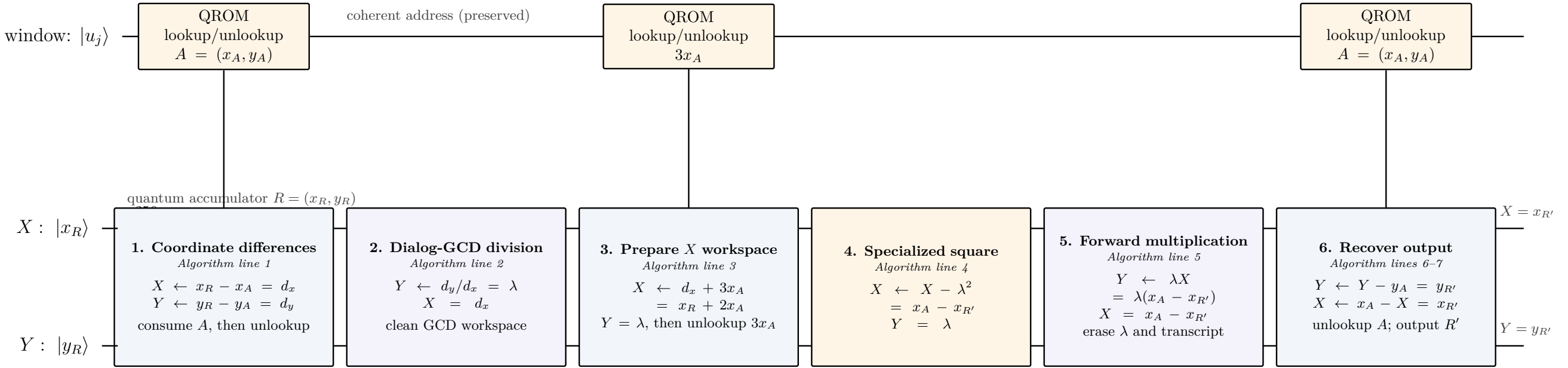
A_j is not an unexplained long-lived register: its coordinates are temporary QROM work data and are returned to zero. ECDSA.Fail supplies A classically; full Shor must validate the coherent interface.

Notation is consistent across the deck: A is the selected addend, R is the input accumulator, and R'

$= R + A$ is the updated accumulator.

From the point-addition algorithm to the 8e9c9a2 submission

Example: $+[u_1 2^{16}]P$, with selected addend $A = [u_1 2^{16}]P$ and output $R' = R + A$



Live quantum-register state											
	DIFFERENCES		DIVISION		PREPARE		SQUARE		MULTIPLY		OUTPUT
X register	$x_R - x_A$	→	$x_R - x_A$	→	$x_R + 2x_A$	→	$x_A - x_{R'}$	→	$x_A - x_{R'}$	→	$x_{R'}$
Y register	$y_R - y_A$	→	λ	→	λ	→	λ	→	$\lambda(x_A - x_{R'})$	→	$y_{R'}$

Source phases in submission 8e9c9a2 (same columns as the pipeline)					
Coordinate differences t1m_coord_x_sub t1m_coord_y_sub	Dialog-GCD division t1m_inverse	Prepare X workspace t1m_coord_add3x	Specialized square t1m_square	Forward multiplication t1m_forward_multiply	Recover output t1m_coord_y_sub_final t1m_coord_rsub_final

Pipelines 2 and 5 share the dialog-GCD transcript machinery: the first produces λ , while the second replays it to multiply by X and erase the transcript.

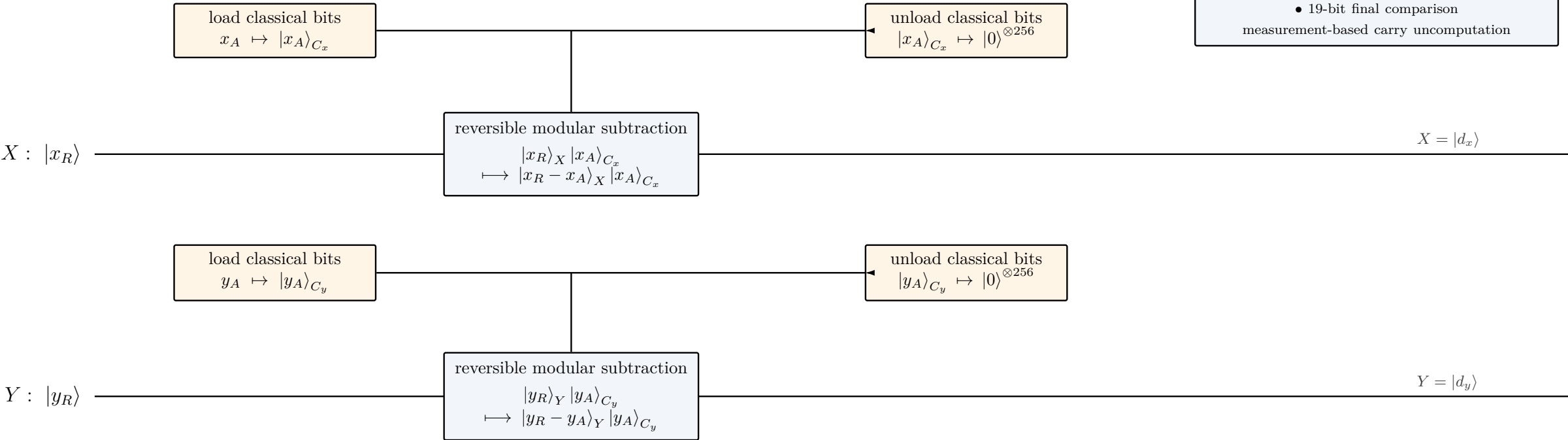
In ECDSA.Fail, (x_A, y_A) are classical input bits; a full windowed-Shor circuit must separately implement and validate the coherent QROM interface.

Phase 1: form the coordinate differences

$$d_x = x_R - x_A \pmod{p}, \quad d_y = y_R - y_A \pmod{p}$$

Phase 1 optimization stack

- off-peak classical addend
- serialized 256-qubit workspace
- 53-bit pseudo-Mersenne fold
- 19-bit final comparison
- measurement-based carry uncomputation



Reversible-state summary

$$|x_R\rangle_X |y_R\rangle_Y |0\rangle_{C_x} |0\rangle_{C_y} \mapsto |d_x\rangle_X |d_y\rangle_Y |0\rangle_{C_x} |0\rangle_{C_y}$$

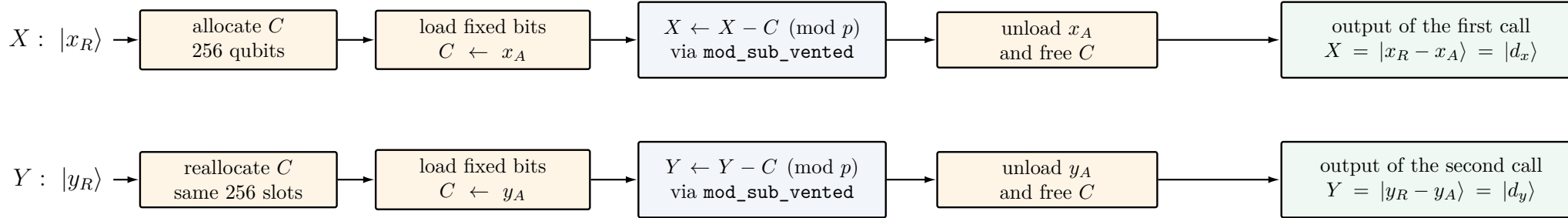
The two coordinate subtractions are independent. The classically known addend is materialized only for the duration of each arithmetic block and is then removed, so no copy of (x_A, y_A) remains live.

Source correspondence: `tlm_coord_x_sub` and `tlm_coord_y_sub` in submission `8e9c9a2`.

Phase 1 detail: optimized coordinate subtraction

The group-law step is conventional; the implementation reduces its live width and nonlinear cost.

Serialized use of one transient coordinate register



Classical-addend specialization

For a classical input bit c_i , X^{c_i} tells the controller to apply X to the target C_i iff $c_i = 1$:

$$|0\rangle_{C_i} \xrightarrow{X^{c_i}} |c_i\rangle_{C_i}.$$

Thus $X^0 = I$ and $X^1 = X$; c_i is not a quantum input wire. The loaded coordinate $c \in \{x_A, y_A\}$ supplies the operand for

$$|r\rangle |c\rangle_C \mapsto |r - c \bmod p\rangle |c\rangle_C.$$

Repeating the same operations unloads C because $(X^{c_i})^2 = I$. The x_A and y_A calls are sequential, so only one coordinate copy is live.

Pseudo-Mersenne modular correction

For $0 \leq x, y < p$, computing r_0 generates internal borrows $b_1, \dots, b_{255}$ and the final borrow $b = b_{256} = [y < x]$:

$$r_0 = (y - x) \bmod 2^{256} = \begin{cases} y - x, & y \geq x, \\ 2^{256} + y - x, & y < x. \end{cases}$$

For $F = 2^{32} + 977$, the pseudo-Mersenne identity gives

$$p = 2^{256} - F \equiv 0 \pmod{p} \implies 2^{256} \equiv F \pmod{p}.$$

$$r = (y - x) \bmod p = \begin{cases} r_0, & y \geq x, \\ r_0 - F = p + y - x, & y < x, \end{cases}$$

Thus $r = r_0 - bF \pmod{2^{256}}$. The source corrects $\text{low}_{53}(r_0)$ and measurement-uncomputes b using a 19-MSB phase predicate. Equal top windows can depend on lower bits; these truncations are benchmark-validated rather than all-input proofs.

Measurement-based carry uncomputation “carry venting” in the implementation

Selected internal borrows b_i , rather than the final $b = b_{256}$, are “vented.” A forward Toffoli stores an internal carry $c = q_1 \wedge q_2$ in a clean ancilla:

$$\sum_{q_1, q_2} \alpha_{q_1 q_2} |q_1, q_2\rangle |0\rangle \mapsto \sum_{q_1, q_2} \alpha_{q_1 q_2} |q_1, q_2\rangle |q_1 q_2\rangle.$$

Measuring the ancilla in the X basis with result $m \in \{0, 1\}$ leaves

$$\sum_{q_1, q_2} \alpha_{q_1 q_2} (-1)^{mq_1 q_2} |q_1, q_2\rangle.$$

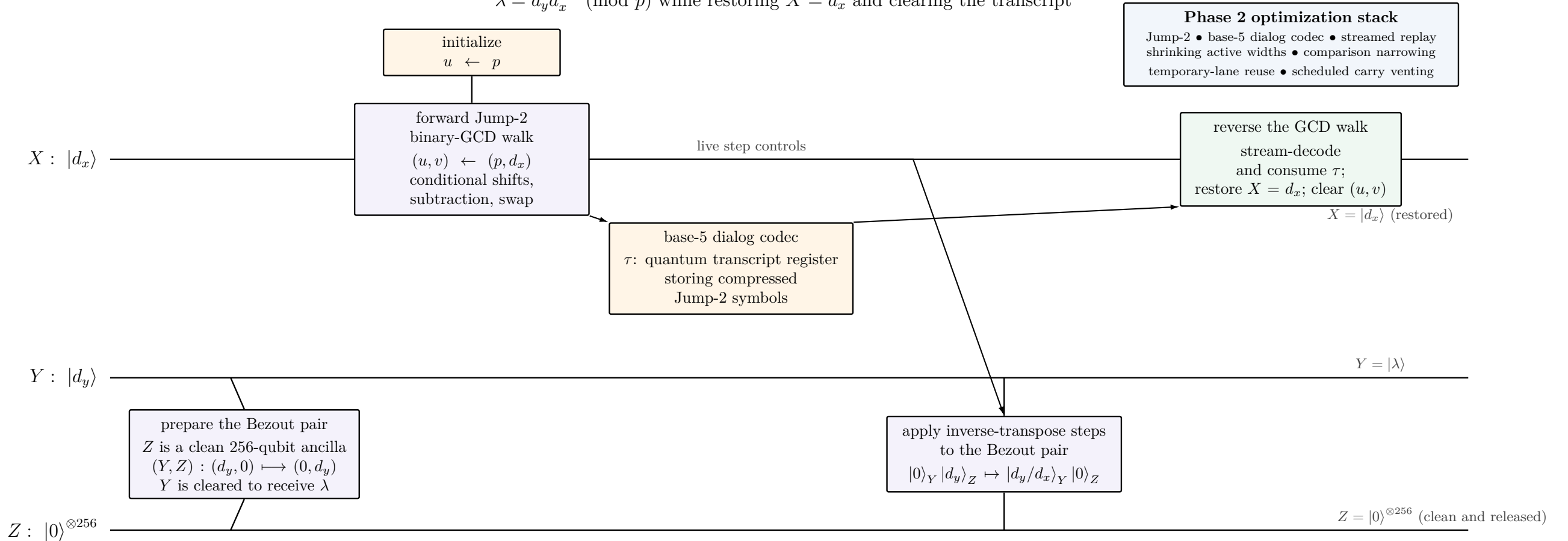
The correction CZ_{q_1, q_2}^m removes the phase exactly, omitting the reverse Toffoli. The final b remains live through $-bF$ and is cleaned separately.

Interpretation. Phase 1 does not introduce a new point-addition identity. Its savings come from source-level liveness control, arithmetic specialized to p , and measurement-assisted uncomputation. The classical loading shown here is valid for the challenge’s fixed-addend interface; a coherent windowed-Shor implementation instead requires a separately validated QROM lookup/use/unlookup construction.

Source correspondence: the sequential `coord_addsub` calls and `mod_sub_vented` implementation in submission `8e9c9a2`.

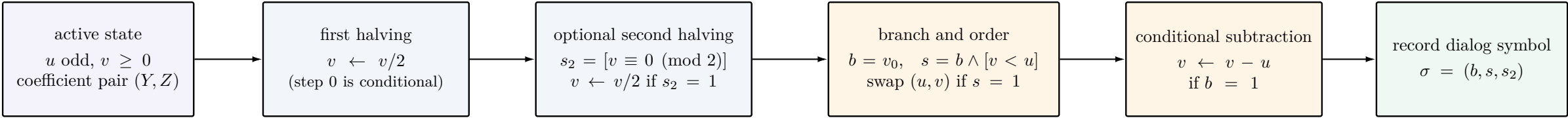
Phase 2: compute the slope by dialog-GCD division

$\lambda = d_y d_x^{-1} \pmod{p}$ while restoring $X = d_x$ and clearing the transcript



Phase 2 detail: the Jump-2 Euclidean step

Up to two binary-GCD halvings are represented by one reversible control symbol



Only five symbols are reachable

$s_2 = 0 \Rightarrow b = 1$, because the value remaining after the first halving is odd;
 $b = 0 \Rightarrow s = 0$, because swapping is enabled only on a subtraction step.

(1, 0, 0)

(1, 1, 0)

(0, 0, 1)

(1, 0, 1)

(1, 1, 1)

Why the recorded symbol is sufficient

The walk applies a data-dependent linear update M to (u, v) , where v is physically the coordinate register X . The paired arithmetic applies M^{-T} directly to physical registers (Y, Z) , preserving the division relation. The bit s_2 selects whether this update contains one or two powers of two, while b and s select subtraction and operand order.

$$\text{walk: } (u, v) \xrightarrow{M} (u', v') \quad \Longleftrightarrow \quad \text{physical pair: } (Y, Z) \xrightarrow{M^{-T}} (Y', Z')$$

Compared with a one-step binary-GCD schedule, Jump-2 packages the mandatory halving and a possible second halving into one recorded iteration. Sub-mission **8e9c9a2** executes 261 such iterations and replays their symbols in reverse order to recover the quotient while restoring the Euclidean workspace.

Implementation correspondence: forward_gcd_jump, apply_step_reverse, apply_step_forward, and reverse_gcd_jump.

Phase 2 detail: base-5 codec and streamed dialog replay

Compress the five-way Jump-2 control alphabet, then decode only the symbols needed by the active replay step

1. Reachable support

A raw Jump-2 symbol has three bits, but only five values occur:

100

110

001

101

111

$$\phi : \{\sigma\} \longrightarrow \{0, 1, 2, 3, 4\}$$

Here ϕ denotes any fixed bijection; the circuit implements it as a reversible Boolean network.

2. Pack three symbols

$$d_j = \phi(\sigma_j), \quad z = d_0 + 5d_1 + 25d_2$$

$$5^3 = 125 < 2^7$$

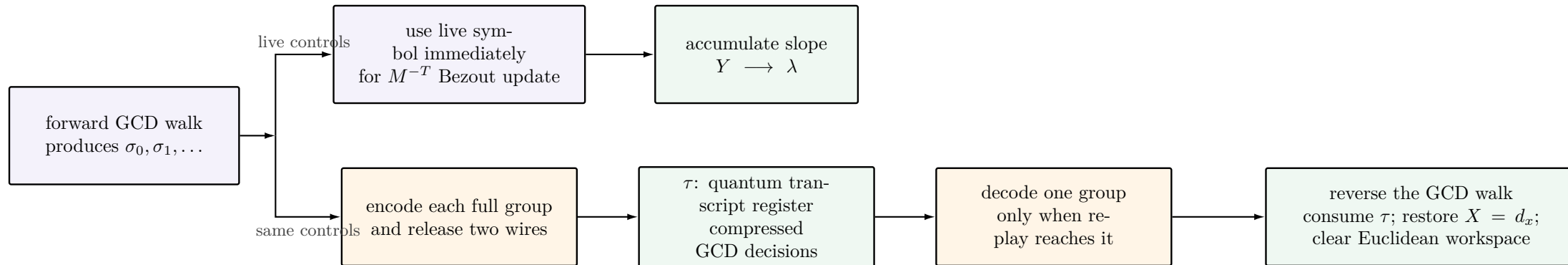
$3 \times 3 = 9$ raw qubits
 $\longrightarrow 7$ encoded qubits

3. Tape layout

For 261 scheduled symbols, the restored submission uses

$$\underbrace{2}_{\text{step 0}} + \underbrace{86 \cdot 7}_{\text{triple blocks}} + \underbrace{5}_{\text{final pair}} = 609$$

qubits, versus $261 \cdot 3 = 783$ for naive raw storage. Boundary symbols use specialized two- and five-bit layouts.



The width reduction comes from scheduling as well as encoding

shrinking Euclidean registers u, v

growing τ

remaining u, v

compressed tape τ

As the active Euclidean registers narrow, τ grows into the released space. Streamed decoding consumes its symbols during reverse replay, restores the Euclidean workspace, and returns the transcript ancillas to $|0\rangle$.

Source terminology: this slide describes `DialogCodec::Pair` and `DialogCodec::Triple`; related source paths and development notes use the “K5 codec” jargon for the corresponding five-symbol encoding idea.

Phase 2 detail: width scheduling and qubit reuse

The GCD state becomes smaller as the dialog grows, and short-lived workspaces are fitted into the released lanes

Scheduled active widths

$$w_i = \text{SCHED_J2}[i]$$

At iteration i , the implementation retains only the active w_i -bit slices of u and v . High-order lanes whose values are known to be zero are released, so shifts, swaps, and conditional subtraction become progressively narrower.

shrinking (u, v) creates
room for the growing tape τ

Truncated comparison windows

$$k_i = \text{cmp_window}(i, w_i) < w_i$$

The swap decision compares only a scheduled portion of the active operands. This reduces comparator work and scratch, but the selected windows are calibrated under the benchmark’s validation model rather than justified here as an all-input exact replacement.

smaller comparison window
 $\Rightarrow$ fewer gates and ancillas

Temporary hosting and cleanup

The known bits $u_0 = 1$ and $v_0 = 0$ can be parked while dormant, exposing their physical lanes to temporary state. Already-live registers also serve as dirty “vent” space for chunked adders, with every borrowed value restored before its owner becomes active again.

Measurement-assisted uncomputation clears selected AND and carry ancillas using classical phase feedback, shortening their ownership intervals without changing the logical map.

Combined live-width model

$$Q_i \approx \underbrace{512}_{\text{Bezout pair}} + \underbrace{|\tau_i|}_{\text{compressed dialog}} + \underbrace{2w_i}_{u_i, v_i} + \underbrace{q_{\text{work}, i}}_{\text{comparison, carry, and control}}$$

The central scheduling objective is to make the decrease in $2w_i$ absorb the growth of $|\tau_i|$, while hosting and measurement-based cleanup keep $q_{\text{work}, i}$ below the same peak.

Exact reversible mechanisms include support-set encoding, streamed replay, known-bit parking, restoration of borrowed dirty lanes, and phase-corrected measurement uncomputation. The chosen narrowing and comparison schedules carry the finite-support evidential status of the submitted benchmark circuit.

Implementation correspondence: SCHED_J2, cmp_window, park_known_one, park_known_zero, with_dirty_vent_pool, and clear_and.

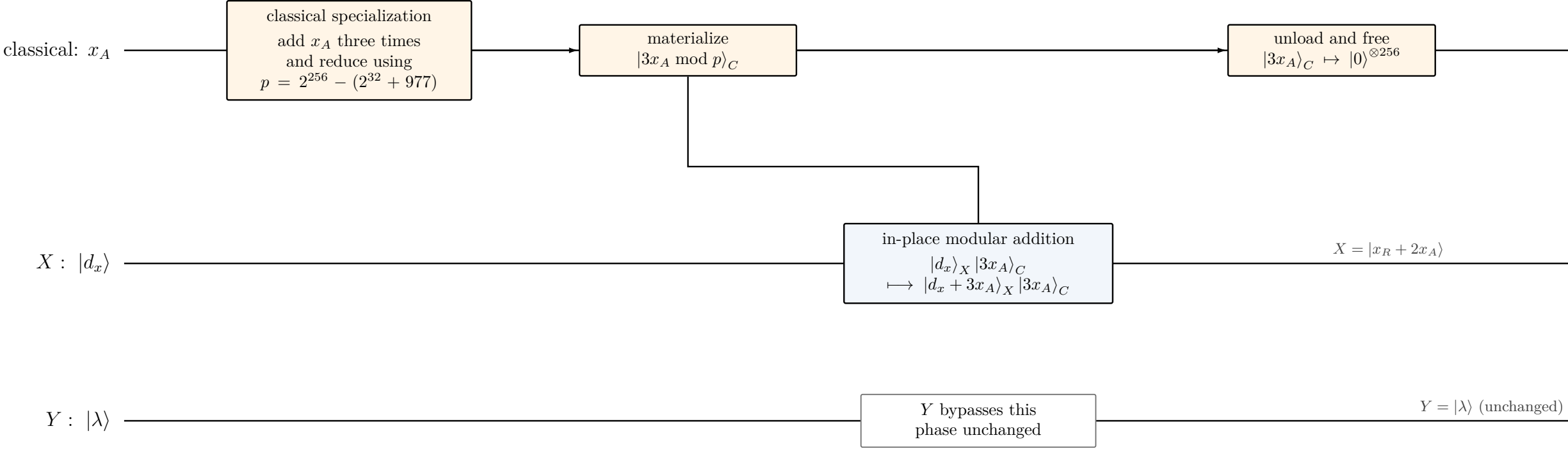
Phase 3: prepare the X workspace

$X = d_x \mapsto d_x + 3x_A = x_R + 2x_A \pmod p, \quad Y = \lambda$

Phase 3 optimization stack

classical $3x_A \bmod p$ • transient 256-qubit materialization

53-bit fold • 19-bit comparison • immediate unload



Why the transformation prepares the square step

$d_x + 3x_A = (x_R - x_A) + 3x_A = x_R + 2x_A$

The following phase subtracts λ^2 , producing $x_R + 2x_A - \lambda^2 = x_A - x_{R'}$. The submitted route uses its configured low-cost modular-adder variant; the classical multiplier does not occupy quantum storage outside this phase.

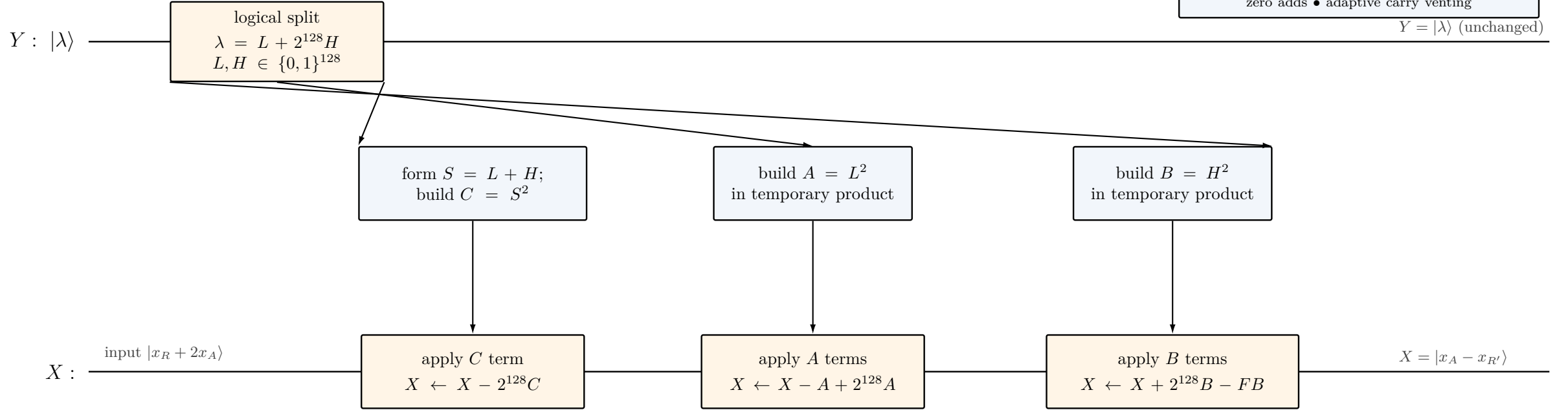
Source correspondence: *tlm_coord_add3x*, *classical_times3_mod_q*, and *coord_add3x*.

Phase 4: subtract the specialized modular square

$$X \mapsto X - \lambda^2 = x_A - x_{R'} \pmod{p}, \quad p = 2^{256} - F, \quad F = 2^{32} + 977$$

Phase 4 optimization stack

three-square Karatsuba • symmetric squaring • streamed products
pseudo-Mersenne NAF fold • from-zero adds • adaptive carry venting



Identity implemented by the streamed square

$$\lambda^2 = A + 2^{128}(C - A - B) + 2^{256}B \equiv A + 2^{128}(C - A - B) + FB \pmod{p}$$

The signed updates shown above sum to $-\lambda^2$. Each of A , B , and C is built, accumulated into X , and reversibly unbuilt before the next product is retained; $Y = \lambda$ is never overwritten.

The reduction of the $2^{256}B$ term to FB exploits the pseudo-Mersenne form of the secp256k1 prime and avoids a generic full-width modular-square workspace.

Source correspondence: *mod_square_sub_pm_secp256k1_symmetric*, including the C , A , and B build/apply/unbuild phases.

Phase 4 detail: symmetry-aware streamed Karatsuba squaring

A standard one-level Karatsuba identity is specialized to squaring, reversible streaming, and the secp256k1 prime.

Standard Karatsuba square identity

Let $\beta = 2^{128}$ and write $\lambda = L + \beta H$. To avoid confusion with the selected elliptic-curve addend A , denote the three temporary squares by U , V , and W :

$$U = L^2, \quad V = H^2, \quad W = (L + H)^2.$$

$$W - U - V = 2LH, \quad \lambda^2 = U + \beta(W - U - V) + \beta^2 V.$$

$$p = 2^{256} - F, \quad F = 2^{32} + 977 \implies \beta^2 = 2^{256} \equiv F \pmod{p}.$$

$$\therefore \lambda^2 \equiv U + \beta(W - U - V) + FV \pmod{p}.$$

Karatsuba supplies the algebra; the circuit-specific gains come from symmetric partial products, immediate accumulation, and pseudo-Mersenne folding.

W contribution
 $X \leftarrow X - \beta W$

U contributions
 $X \leftarrow X - U + \beta U$

V contributions
 $X \leftarrow X + \beta V - FV$

Reversible build–apply–unbuild schedule

1. Split and prepare. View $Y = \lambda$ as the 128-bit slices L and H ; reversibly build the 129-bit sum $S = L + H$.

2. Sum square. Build $W = S^2$, apply $-\beta W$ directly to X , and reverse the squarer to return its product register to zero.

3. Low-half square. Build $U = L^2$, apply $-U + \beta U$ directly to X , and unbuild U .

4. High-half square. Build $V = H^2$, apply $+\beta V - FV$ directly to X , and unbuild V .

5. Clean the preparation. Reverse the addition that formed $S = L + H$; all square work registers return to $|0\rangle$.

Result: $Y = \lambda$ is preserved and $X_{\text{out}} = X_{\text{in}} - \lambda^2 = x_R + 2x_A - \lambda^2 = x_A - x_{R'} \pmod{p}$.

Circuit-level specializations beyond standard Karatsuba

Symmetric squaring

Diagonal bits are copied directly. The three squares form 24,512 unordered cross products, 24.9% fewer than a direct 256-bit symmetric square.

Direct streamed accumulation

Only one of U , V , or W is live at a time. Each signed contribution is applied to X and immediately uncomputed, avoiding a persistent 512-bit square.

Pseudo-Mersenne fold

The high limb $2^{256}V$ becomes $FV \pmod{p}$, replacing generic full-width modular reduction by a fixed-constant fold.

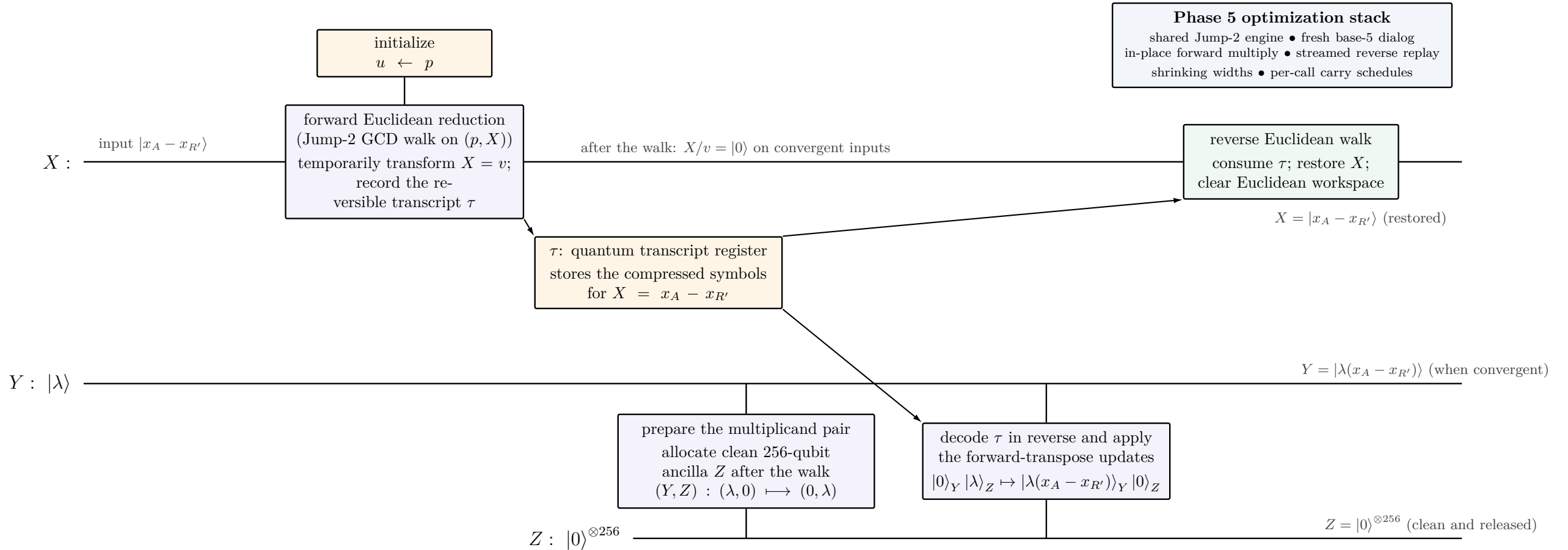
Sparse signed constant

$F = 2^{32} + 2^{10} - 2^6 + 2^4 + 1$, so FV is assembled from five shifted additions or subtractions rather than a generic multiplication.

The Karatsuba identity is exact; any benchmark-calibrated finite-width correction is a separate optimization. Source correspondence: `mod_square_sub_pm_secp256k1_symmetric`.

Phase 5: forward multiplication via dialog-GCD replay

$$Y = \lambda \mapsto \lambda(x_A - x_{R'}) \pmod{p} \text{ while restoring } X = x_A - x_{R'}$$



Why dialog-GCD implements multiplication in Phase 5

Phase 2 realizes the in-place division unitary

$$U_{\text{div}} : |X\rangle |Y\rangle \mapsto |X\rangle |YX^{-1} \bmod p\rangle.$$

Reversibility therefore gives

$$U_{\text{div}}^{-1} : |X\rangle |Y\rangle \mapsto |X\rangle |YX \bmod p\rangle.$$

Thus Phase 5 reuses the low-space division construction in reverse; it does not compute X^{-1} .

Here $X = x_A - x_{R'}$. Phase 2's transcript described the different operand $d_x = x_R - x_A$ and was consumed during cleanup. Phase 5 therefore constructs a fresh $\tau(X)$, whose reverse replay supplies the X -dependent coefficient updates that transform λ into λX while restoring X .

Resource effect. The in-place map reuses X , Y , Z , and τ , avoiding the additional product register and cleanup required by a straightforward out-of-place modular multiplication. This lowers peak-width pressure, but does not establish a Toffoli advantage: transcript replay remains a dominant gate cost. Cleanup is exact; the product relation requires 261-step convergence, validated only on the benchmark support.

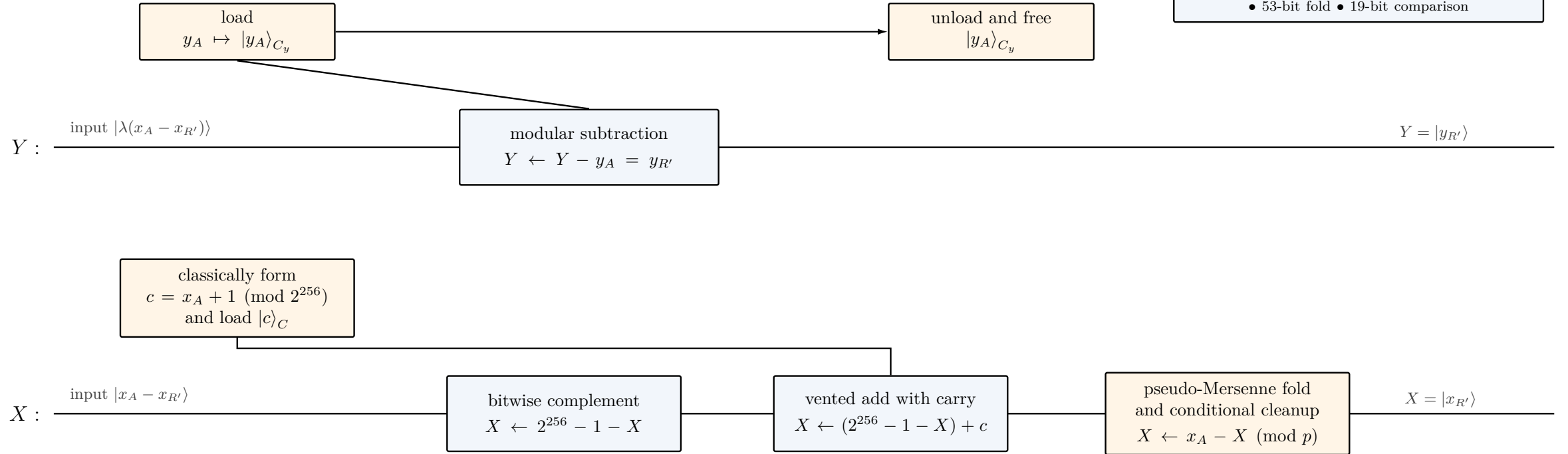
Source correspondence: `mod_mul_inverse_in_place(..., Direction::Forward)`, `ITERS=261`, and reverse replay with `apply_step_forward`.

Phase 6: recover the affine output coordinates

$$y_{R'} = \lambda(x_A - x_{R'}) - y_A, \quad x_{R'} = x_A - (x_A - x_{R'})$$

Phase 6 optimization stack

transient classical coordinates • fused
complement-plus-add subtraction
measurement-based carry cleanup
• 53-bit fold • 19-bit comparison



Final reversible map

$$|x_A - x_{R'}\rangle_X |\lambda(x_A - x_{R'})\rangle_Y \mapsto |x_{R'}\rangle_X |y_{R'}\rangle_Y$$

The fused reverse subtraction uses $\overline{X} + (x_A + 1) = x_A - X \pmod{2^{256}}$ and then applies the secp256k1 modular correction.
All loaded classical coordinates, carry state, and temporary reduction data are removed before returning the two quantum output registers.

Source correspondence: `t1m_coord_y_sub_final`, `t1m_coord_rsub_final`, and the enabled `fused_mod_rsub_vented_loaded` route.